

One Prefill Prices a Pool of Reasoning Models

Anonymous authors

Abstract

Reasoning models generate long, query-dependent outputs, making model-average length an unreliable basis for pricing individual requests. We predict the costs of an entire reasoning-model pool from one frozen 4B encoder’s prefill activations. Linear output-length heads reuse the representation used for success prediction; the target models need not expose their activations or run before selection. Holding success predictions fixed, this cross-model cost readout saves 35.6% on LiveCodeBench, 21.9% on Omni-MATH, and 30.7% on MMLU-Pro relative to median-length pricing at matched accuracy. It yields larger savings than our embedding-based and prompt-feature estimators on these pools, although the differences on Omni-MATH are inconclusive. A dedicated cost head adds 21.5 percentage points over success-derived pricing on MMLU-Pro, where subject and required work carry cost information beyond correctness probability. Other coding pools have substantial oracle headroom but no significant learned gain. One shared prefill can therefore price heterogeneous reasoning routes effectively, with workload-dependent limits.

1. Introduction

A router choosing among reasoning models must estimate both answer quality and generation cost. Token prices are known, but reasoning-trace lengths vary greatly across queries. A model that is cheap on average may be expensive on a particular problem. Predicting query–model costs before generation can change which model offers the best trade-off.

Learned cost is already part of routing. MixLLM predicts output length from query embeddings [9]; CARROT uses embedding-based nearest neighbors or a fine-tuned RoBERTa encoder to estimate cost and accuracy [7]. CARROT also compares against constant pricing and reports marginal improvements on its benchmarks. The architectural question we study is whether *one frozen generative model’s prefill can price an entire pool of reasoning models* and provide more useful routing signals than our implemented text-based estimators.

We attach per-route output-length heads to the same pre-

fill representation used for success prediction. One 4B encoder processes the query; simple ridge readouts estimate the costs of other models and effort settings. This requires neither target-model activations nor a target generation before choosing the route. For an existing prefill router, cost prediction adds readouts without another encoder pass.

Our contribution combines this cross-model cost readout with a controlled evaluation: success predictions and the routing objective stay fixed while pricing changes. Three reasoning workloads yield 22–36% savings against median-length pricing. Dedicated cost heads are especially useful on MMLU-Pro, where subject and item-level work provide information beyond success probability. Oracle headroom and negative coding results identify limits: available savings need not be predictable from the prompt.

Related work. SharedTrunkNet uses encoder–target decoupling to predict other models’ correctness from prefill activations, but prices outputs by median training length [8]. We extend this architecture with cross-model cost readouts. Activation-based length prediction also exists for inference scheduling [6, 11]; our focus is one shared encoder pricing multiple reasoning routes before dispatch. CARROT’s shared representation and separate cost/performance outputs are close architecturally [7]; we use frozen generative activations with linear readouts. ZeroRouter derives length from item-response difficulty bins [12]; we test whether pricing needs information beyond the success signal.

2. Cross-Model Cost from One Prefill

For query x and route m , let $p_m(x)$ be the probability of a correct answer and $c_m(x)$ its expected generation cost. A route denotes a model and a fixed reasoning-effort setting. The router selects

$$m^*(x; V) = \arg \max_m \{V \hat{p}_m(x) - \hat{c}_m(x)\}, \quad (1)$$

where V controls the value assigned to a correct answer. We commit to one route before generation, with no correctness verifier or fallback call.

Shared features, separate readouts. A frozen Qwen3-4B encoder processes the prompt once. We concatenate

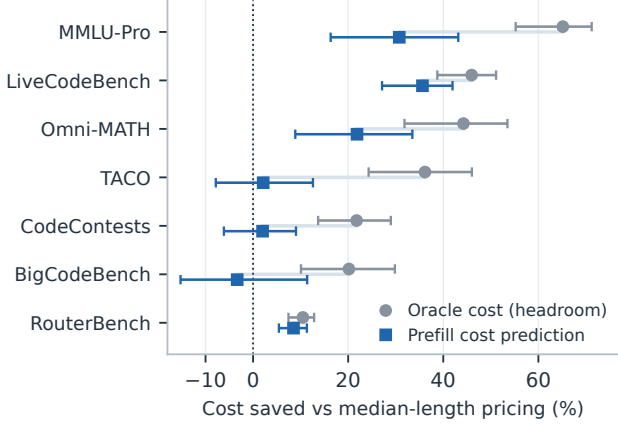


Figure 1. **Available savings need not be captured.** Oracle and predicted cost replace median output length with the success head held fixed. Whiskers show 95% paired-bootstrap intervals. Reasoning pools have 20–65% oracle headroom, but learned gains vary substantially; the chat contrast has less headroom. Each pool uses its own shared accuracy band.

mean-pooled and last-token activations from eight stored layers, then standardize features using training statistics. The Instruct-2507 encoder is used on LiveCodeBench and MMLU-Pro; the Thinking-2507 variant is fixed for Omni-MATH. Each route has a regularized logistic success head, fitted to valid draw outcomes and Platt calibrated on held-out data. A separate ridge head predicts the log of mean output tokens from the same representation. Its regularization is selected using training-only cross-validation. Each cost head is trained on its target route’s output lengths; no zero-shot transfer to unseen models is assumed. The heads share features without forcing success and length through a common scalar latent.

From length to price. Let $\hat{\ell}_m(x)$ denote predicted output tokens after exponentiating the log-length prediction, applying a training-residual smearing correction, and matching the route’s training mean. For input tokens $n_m^{\text{in}}(x)$ and input/output rates $r_m^{\text{in}}, r_m^{\text{out}}$ in dollars per million tokens,

$$\hat{c}_m(x) = \frac{r_m^{\text{in}} n_m^{\text{in}}(x) + r_m^{\text{out}} \hat{\ell}_m(x)}{10^6}. \quad (2)$$

Input length is observed before generation. The reference replaces $\ell_m(x)$ with the median valid training output length for route m . The empirical oracle uses the problem–route’s mean observed output length. Only the oracle sees evaluation output lengths when deciding; all arms are charged observed tokens after their routing decisions are fixed.

3. Experimental Protocol

Our main pools cover coding (LiveCodeBench [3], 892 problems), olympiad math (an Omni-MATH [1] subset, 500), and knowledge/reasoning (MMLU-Pro [10], 1,000). Train/calibration/test counts are 441/110/341, 275/75/150, and 550/150/300, respectively. Their five routes are gpt-oss-20b at low and medium effort, deepseek-v4-flash, and gpt-oss-120b at medium and high effort. We retain repeated valid draws: approximately 16/12/8/6/3 per route on LiveCodeBench and 4/3/3/2/2 on the other main pools. Correctness and cost for each problem–route are estimated from these draws. We also analyze CodeContests [5], TACO [4], BigCodeBench [13], and the 11-model RouterBench [2] chat pool. Contrast pools use their recorded configurations, rather than necessarily the main five-route pool.

Pricing and controls. All main comparisons use recorded market rates: input/output dollars per million tokens are 0.018/0.09 for gpt-oss-20b, 0.04704/0.09408 for deepseek-v4-flash, and 0.15/0.60 for gpt-oss-120b. Prices are held fixed across arms. The prefill is common to the success-only and success-plus-cost router; reported savings isolate target generation spend. They are incremental savings for an existing prefill router, rather than a claim that deploying the encoder is free.

Matched-accuracy evaluation. We sweep V , construct test accuracy–cost frontiers, and allow randomized mixtures between convex-hull points. For 12 equally spaced targets in the interior of the accuracy range shared by the compared arms, savings are

$$G = 1 - \exp \left[\frac{1}{12} \sum_{j=1}^{12} \log \frac{C_{\text{arm}}(a_j)}{C_{\text{median}}(a_j)} \right]. \quad (3)$$

Oracle G is headroom; learned G is captured savings. We use 500 paired bootstrap resamples of test problems, keeping all routes and draws of a problem together. These are descriptive test-frontier comparisons: their operating points are not prospectively selected. Separate calibration-selected checks evaluate deployment, with achieved test accuracy reported independently.

4. Results: Cross-Model Pricing and Its Limits

One shared prefill captures substantial savings. Changing only pricing reduces target generation spend by 22–36% on the three main pools (Table 1), with all three paired intervals excluding zero. The learned heads capture approximately 47–78% of their oracle headroom (46.0 / 44.2 / 65.1%; Fig. 1). Savings arise without replacing the success architecture or changing the routing objective.

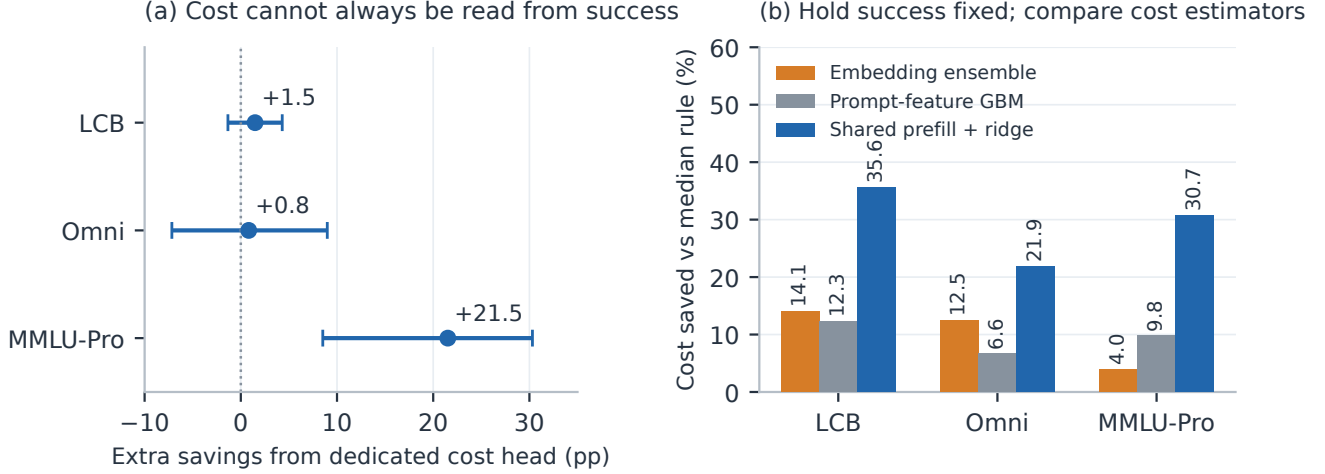


Figure 2. **Prefill activations supply useful cross-model cost signals.** (a) Added savings over cost derived from success; whiskers are paired 95% intervals. (b) Our shared-prefill heads versus adapted embedding and prompt-feature estimators, with success predictions held fixed. Bars show point estimates; paired differences are reported in the text. Panels show percentage-point differences and absolute percentage savings, respectively.

Baseline (its own gain: LCB / Omni / MMLU-Pro)	LCB	Omni	MMLU-Pro
Ours (gain vs. median rule)	35.6 [27.1, 41.9]	21.9 [8.9, 33.5]	30.7 [16.3, 43.2]
Mean per-model constant (1.3 / -1.4 / 1.2)	+34.3 [†]	+23.3 [†]	+29.5 [†]
Single best model (-3.7 / -20.1 / 5.4)	+39.3 [†]	+42.0 [†]	+25.3 [†]
MixLLM-style embedding ensemble (14.1 / 12.5 / 4.0)	+21.5 [†]	+9.4 [-4.4, 24.2]	+26.7 [12.4, 40.2]
Prompt-feature GBM (12.3 / 6.6 / 9.8)	+23.3 [†]	+15.2 [-1.4, 34.1]	+20.9 [3.8, 33.3]
Cost from the success head (34.1 / 21.0 / 9.2)	+1.5 [-1.3, 4.3]	+0.8 [-7.2, 9.0]	+21.5 [8.5, 30.3]
ZeroRouter-style bins on our difficulty, K=5 (32.2 / 21.7 / 10.9)	+3.7 [0.1, 7.4]	-0.1 [-12.1, 10.7]	+19.6 [7.8, 32.7]
ZeroRouter-style bins on our difficulty, K=10 (34.0 / 17.9 / 13.3)	+1.8 [-1.6, 5.1]	+3.5 [-8.2, 14.9]	+19.1 [7.6, 31.6]
ZeroRouter faithful, tuned on calibration (18.5 / 20.0 / 21.6)	+17.6 [9.8, 24.9]	+3.0 [-10.7, 16.6]	+9.3 [-2.6, 21.5]
ZeroRouter faithful, own DistilBERT encoder (12.7 / 13.0 / 12.1)	+23.2 [†]	+9.8 [†]	+18.8 [†]

Table 1. Canonical baselines at matched accuracy (test): ours minus the baseline’s cost saved vs. the median-length rule (baseline’s own gain in parentheses); brackets are paired 95% intervals; [†] = unpaired differences of reported gains. “Bins on our difficulty” is ZeroRouter-style bin pricing on our shared-difficulty signal; “faithful” reproduces ZeroRouter’s IRT stage with configuration chosen on calibration (ZR rows use that comparison’s band: ours 36.0 / 22.7 / 30.9).

Comparison with text-based estimators. Our embedding baseline averages per-route MLP, random-forest, and nearest-neighbor regressors on jina-code embeddings, adapting MixLLM’s predictor family. A gradient-boosted baseline uses prompt length, numbers, examples, constraints, and keyword counts. Both predict log output length with the same training split and price conversion as our heads; they are adapted estimators, not reproductions of complete published routers. Table 1 gives savings and paired differences: on MMLU-Pro, paired advantages are 26.7 points [12.4, 40.2] and 20.9 [3.8, 33.3]; on Omni they are inconclusive (9.4 [-4.4, 24.2] and 15.2 [-1.4, 34.1]). The representation improves routing on important workloads without a uniform superiority claim.

Large variation is an opportunity, not a guarantee. Figure 1 exposes substantial headroom on TACO (36.1%), CodeContests (21.8%), and BigCodeBench (20.2%), but learned gains of 2.1%, 2.0%, and -3.3% have intervals spanning zero; RouterBench’s headroom is only 10.5%. Reasoning workloads can benefit considerably more from query-dependent pricing than this chat contrast, yet reasoning alone does not imply a learnable improvement.

The opportunity depends on variation *relative to model price gaps*. If one model is much cheaper on nearly every query, refining output length rarely changes the selected route. Large within-model variation makes cost prediction potentially consequential; its relevance still depends on the pool’s prices and success trade-offs.

Difficulty explains some, but not all, useful cost variation. On LiveCodeBench and Omni-MATH, hard problems tend to require long outputs across routes. Between-problem differences account for most log-length variance, and within a problem, unsuccessful draws are not systematically longer than successful ones. Prompt features can therefore predict a shared length level before generation.

To ask whether cost needs its own readout, we price each route using a regression on the success heads’ predicted logits and their squares. This cost-from-success control retains the same success predictions in routing. Its savings are 34.1% on LiveCodeBench and 21.0% on Omni, not significantly different from the dedicated cost head. On MMLU-Pro it captures only 9.2%, versus 30.7% with a dedicated readout: a paired difference of 21.5 percentage points [8.5, 30.3] (Fig. 2). Here even empirical outcome-based difficulty explains little of log length (mean $R^2 \approx .19$). Subject-aware pricing raises savings to 22.3%, suggesting that domain and required work contain cost information beyond probability of success. We interpret required work as a plausible mechanism, not a directly measured causal variable.

Complementary to another success architecture. We reproduce ZeroRouter’s item-response success model and difficulty-bin pricing [12] on our deployment pools, using MAP rather than SVI and frozen 4B features for latent prediction, with dimension, seed, and bin count selected on calibration. End to end, it trails our router by 17.6 points [9.8, 24.9] on LiveCodeBench, with an inconclusive +9.3 [−2.6, 21.5] on MMLU-Pro and a tie on Omni (Table 1). Component swaps locate the difference in pricing rather than success estimation: holding ZeroRouter’s success model fixed, replacing its pricing with our cost heads raises savings from 18.5 to 35.5% on LiveCodeBench, 20.0 to 22.0% on Omni, and 21.6 to 32.4% on MMLU-Pro. Our pricing composes with a second success architecture; the swaps are descriptive point estimates.

Prospective evidence. A pre-registered screen used one gpt-oss-20b-low draw and five-fold cross-validated log-length R^2 to predict at least 10% savings with an interval excluding zero when $R^2 \geq .50$. Omni and MMLU-Pro scores of .67 and .65 correctly predicted gains. Both completed calls predicted gains; negative calls remain untested.

5. Discussion and Limitations

The method separates the model providing the representation from the models being priced. Cost labels remain necessary per route, but query encoding is shared. MMLU-Pro suggests a particular benefit on heterogeneous workloads: adding subject to success-derived pricing recovers roughly 60% of its gap to the full cost probe. This supports

a contribution from domain variation, not a causal explanation of its large headroom. Item-level cost information remains after subject and difficulty are accounted for.

Our main pools share five configurations from two model families. The text-based comparisons use adapted estimators; we do not evaluate CARROT’s exact KNN or fine-tuned RoBERTa implementation. Encoder size, task adaptation, and regression design differ, so these comparisons do not isolate activations alone. Contrast pools differ in candidate models, accuracy ladders, and prices, so their differences are descriptive. Evaluations replay stored draws rather than a live deployed router; empirical oracle costs have sampling error. Bootstrap intervals condition on fitted heads and exclude training and calibration selection uncertainty. Omni has only 150 test problems. Generation savings exclude the common encoder overhead, and deployment-selected operating points can achieve different test accuracies. The ZeroRouter comparison studies a deployment-sized reimplementation, not its original population-scale setting. Finally, the two completed prospective tests cover only predicted gains. Broader model families and prospective negative cases would strengthen generality.

6. Conclusion

One frozen prefill can price a pool of reasoning models: per-route linear cost heads save 22–36% on three workloads while reusing the success encoder. Our implemented text-based estimators capture less of this value. Dedicated pricing is especially useful when success probability does not explain required work, but substantial oracle headroom can remain uncaptured. Cross-model prefill cost prediction is therefore a practical routing component with explicit workload-dependent limits.

References

- [1] Bofei Gao, Feifan Song, Zhe Yang, et al. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark For Large Language Models, 2024. arXiv:2410.07985.
- [2] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, et al. Router-Bench: A Benchmark for Multi-LLM Routing System, 2024. arXiv:2403.12031.
- [3] Naman Jain, King Han, Alex Gu, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code, 2024. arXiv:2403.07974.
- [4] Rongao Li, Jie Fu, Bo-Wen Zhang, et al. TACO: Topics in Algorithmic COde generation dataset, 2023. arXiv:2312.14852.
- [5] Yujia Li, David Choi, Junyoung Chung, et al. Competition-Level Code Generation with AlphaCode, 2022. arXiv:2203.07814.
- [6] Glen Messenger. ALPS: Activation-Based Length Prediction for Intelligent LLM Inference Scheduling, 2026. Zenodo. doi:10.5281/zenodo.19078431.

- [7] Seamus Somerstep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, et al. CARROT: A Cost Aware Rate Optimal Router, 2025. arXiv:2502.03261.
- [8] Tanay Varshney, Annie Surla, Michelle Xu, et al. LLM Router: Rethinking Routing with Prefill Activations, 2026. arXiv:2603.20895.
- [9] Xinyuan Wang, Yanchi Liu, Wei Cheng, et al. MixLLM: Dynamic Routing in Mixed Large Language Models, 2025. arXiv:2502.18482.
- [10] Yubo Wang, Xueguang Ma, Ge Zhang, et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark, 2024. arXiv:2406.01574.
- [11] Huanyi Xie, Yubin Chen, Liangyu Wang, et al. Predicting LLM Output Length via Entropy-Guided Representations, 2026. arXiv:2602.11812.
- [12] Cheng Yan, Wuyang Zhang, Zhiyuan Ning, et al. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space, 2026. arXiv:2601.06220.
- [13] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions, 2025. arXiv:2406.15877.